

Lab 5: Operator Overloading

Theory:

- Define operator overloading. Write down the rules for operator overloading.
- Define friend function. Explain it with the help of example.

Lab:

Question: 1

WAP in C++ to add two-time objects using the concept of operator overloading.

```
#include <iostream>
using namespace std;
class Complex {
private:
    float real;
    float imag;
public:
    Complex(float r = 0, float i = 0) {
        real = r;
        imag = i;
    }
    // Operator overloading of +
    Complex operator + (Complex c) {
        Complex temp;
        temp.real = real + c.real;
        temp.imag = imag + c.imag;
        return temp;
    }
    void display() {
        cout << real << " + " << imag << "i" << endl;
    }
};

int main() {
    Complex c1(2.5, 3.5), c2(1.5, 2.5), c3;
    c3 = c1 + c2; // Using overloaded +
    cout << "Sum: ";
    c3.display();
}
```

```
    return 0;
}
```

Output:

Question: 2

WAP in C++ to add Two Numbers Using Friend Function.

```
#include <iostream>
using namespace std;
class Number {
private:
    int value;
public:
    Number(int v = 0) {
        value = v;
    }
    // Declare friend function
    friend int add(Number a, Number b);
};
// Define friend function
int add(Number a, Number b) {
    return a.value + b.value;
}
int main() {
    Number n1(10), n2(20);
    cout << "Sum = " << add(n1, n2) << endl;
    return 0;
}
```

Output:

Question: 3

WAP in C++ using friend Function Accessing Two Different Classes.

```
#include <iostream>
using namespace std;

class B; // Forward declaration
```

```

class A {
private:
    int a;

public:
    A(int val = 0) {
        a = val;
    }

    // Declare friend function
    friend int add(A, B);
};

class B {
private:
    int b;

public:
    B(int val = 0) {
        b = val;
    }

    // Declare friend function
    friend int add(A, B);
};

// Define the friend function
int add(A objA, B objB) {
    return objA.a + objB.b;
}

int main() {
    A obj1(10);
    B obj2(20);

    cout << "Sum = " << add(obj1, obj2) << endl;

    return 0;
}

```

Output:

Question: 4

Develop a C++ program that illustrates binary operator overloading (+ and -) using a friend function in a class named Distance. This class should store distance in terms of feet and inches. Use friend functions to overload the following operators:

The + operator to perform addition of two Distance objects. The - operator to perform subtraction between two Distance objects.

```
#include<iostream>
using namespace std;
class Distance
{
int feet,inches;
public:
void input( )
{
    cout<<"Enter the values in feet and inches:"<<endl;
    cin>>feet>>inches;
}
void display ( )
{
    cout<<feet<<" Feet and "<<inches<<" Inches"<<endl;
}
    friend void operator +(Distance,Distance);
    friend void operator -(Distance,Distance);
};

void operator +(Distance d1,Distance d2)
{
    int s=(d1.inches+d1.feet*12)+(d2.inches+d2.feet*12);
    int f=s/12;
    int i= s%12;
    cout<<f<<" Feet and"<<i<<" Inches"<<endl;
}

void operator -(Distance d1,Distance d2)
{
    int d=(d1.inches+d1.feet*12)-(d2.inches+d2.feet*12);
```

```

    if(d<0)
        d= -d;
    int f=d/12;
    int i= d%12;
    cout<<f<<" Feet and"<<i<<" Inches"<<endl;
}

int main( )
{
    Distance d1, d2;
    d1.input( );
    d2.input( );
    d1.display( );
    d2.display( );
    cout<<"After the addition of two distance objects result="<<endl;
    d1+d2; //operator+ (d1,d2)
    cout<<"After the subtraction of two distance objects result="<<endl;
    d1-d2; // operator- (d1,d2)
    return 0;
}

```

Output:

Question: 5

Write a C++ program to demonstrate binary operator overloading (+ and -) using member functions for a class named Money that handles values in rupees and paisa.

```

#include<iostream>
using namespace std;
class Money
{
    int rupees, paisa;

public:
    void input( )

```

```

{
    cout<<"Enter the values for rupees and paisa:"<<endl;
    cin>>rupees>>paisa;
}
void display( )
{
    cout<<rupees<<"Rupees and"<<paisa<<"Paisa"<<endl;
}
Money operator +(Money m){
Money temp;
    int s=(paisa+rupees*100)+(m.paisa+m.rupees*100);
    temp.rupees=s/100;
    temp.paisa=s%100;
    return temp;
}
Money operator -(Money m){
    Money temp;
    int d=(paisa+rupees*100)-(m.paisa+m.rupees*100);
    if(d< 0)
        d= -d;
    temp.rupees=d/100;
    temp.paisa=d%100;
    return temp;
}
};
int main( )
{
    Money m1,m2,m3,m4;
    m1.input( );
    m2.input( );
    m1.display( );
    m2.display( );
    m3=m1+m2;//m3=m1.operator+(m2);
    m4=m1-m2;//m4=m1.operator-(m2);
    cout<<"After the addition of two money objects result=";

```

```
m3.display( );  
cout<<"After the subtraction of two money objects result=";  
m4.display( );  
return 0;  
}
```

Output:

Conclusion:

- What did you learn from the lab?